

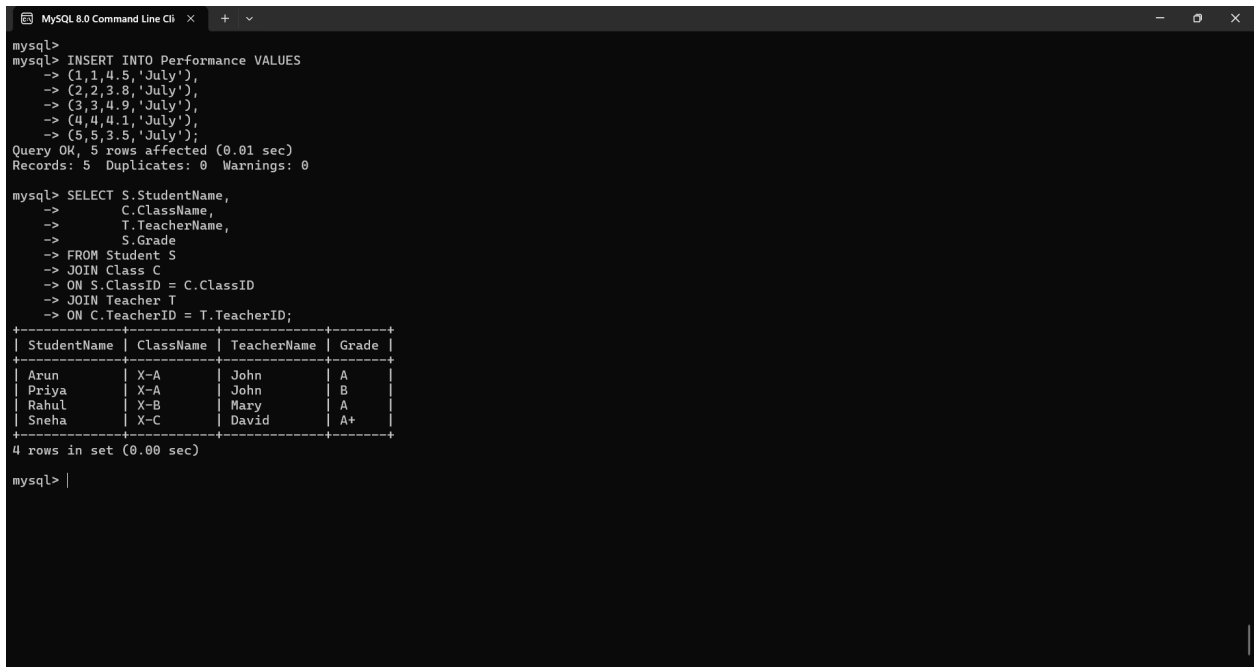
SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

JEEVAN RAO JR
192524495

1. School Management System

```
SELECT S.StudentName,  
       C.ClassName,  
       T.TeacherName,  
       S.Grade  
FROM Student S  
JOIN Class C  
ON S.ClassID = C.ClassID  
JOIN Teacher T  
ON C.TeacherID = T.TeacherID;
```

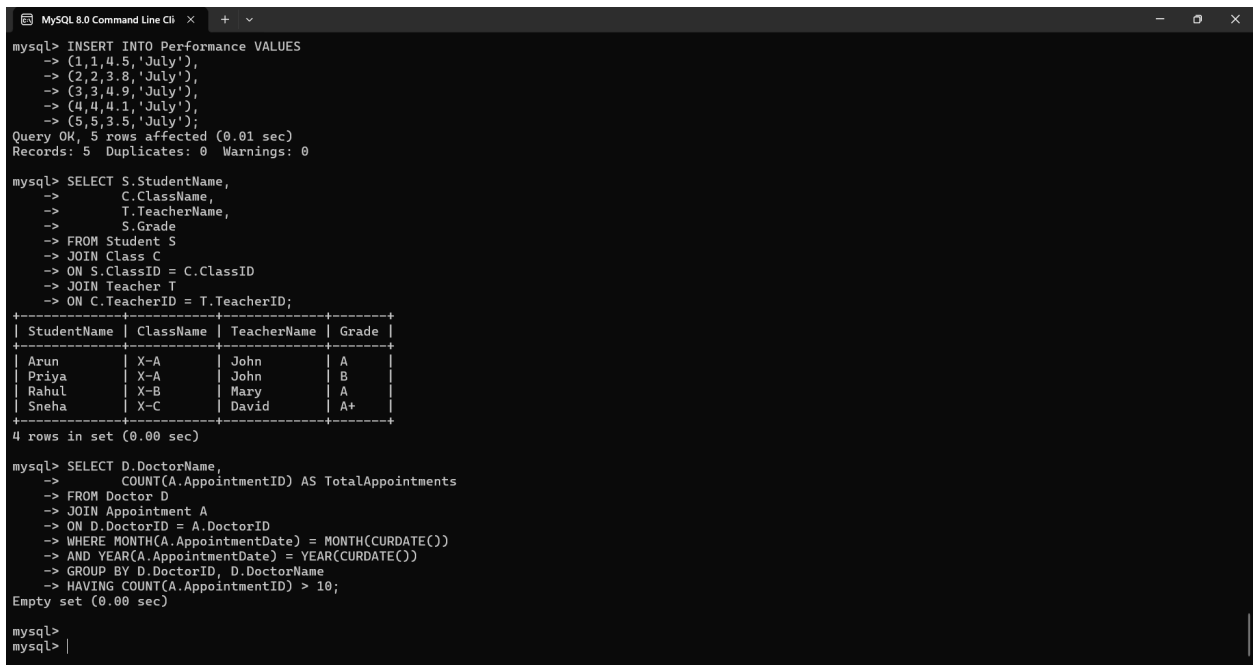


```
mysql> INSERT INTO Performance VALUES  
mysql> --> (1,1,4.5,'July'),  
--> (2,2,3.8,'July'),  
--> (3,3,4.9,'July'),  
--> (4,4,4.1,'July'),  
--> (5,5,3.5,'July');  
Query OK, 5 rows affected (0.01 sec)  
Records: 5 Duplicates: 0 Warnings: 0  
  
mysql> SELECT S.StudentName,  
--> C.ClassName,  
--> T.TeacherName,  
--> S.Grade  
--> FROM Student S  
--> JOIN Class C  
--> ON S.ClassID = C.ClassID  
--> JOIN Teacher T  
--> ON C.TeacherID = T.TeacherID;  
+-----+-----+-----+-----+  
| StudentName | ClassName | TeacherName | Grade |  
+-----+-----+-----+-----+  
| Arun        | X-A      | John        | A     |  
| Priya       | X-A      | John        | B     |  
| Rahul       | X-B      | Mary        | A     |  
| Sneha       | X-C      | David       | A+    |  
+-----+-----+-----+-----+  
4 rows in set (0.00 sec)  
  
mysql> |
```

2. Hospital Appointment System

SQL

```
SELECT D.DoctorName,  
       COUNT(A.AppointmentID) AS TotalAppointments  
FROM Doctor D  
JOIN Appointment A  
ON D.DoctorID = A.DoctorID  
WHERE MONTH(A.AppointmentDate) = MONTH(CURDATE())  
AND YEAR(A.AppointmentDate) = YEAR(CURDATE())  
GROUP BY D.DoctorID, D.DoctorName  
HAVING COUNT(A.AppointmentID) > 10;
```



```
mysql> INSERT INTO Performance VALUES  
-> (1,1,4,5,'July'),  
-> (2,2,3,8,'July'),  
-> (3,3,4,9,'July'),  
-> (4,4,4,1,'July'),  
-> (5,5,3,5,'July');  
Query OK, 5 rows affected (0.01 sec)  
Records: 5 Duplicates: 0 Warnings: 0  
  
mysql> SELECT S.StudentName,  
-> C.ClassName,  
-> T.TeacherName,  
-> S.Grade  
-> FROM Student S  
-> JOIN Class C  
-> ON S.ClassID = C.ClassID  
-> JOIN Teacher T  
-> ON C.TeacherID = T.TeacherID;  
+-----+-----+-----+-----+  
| StudentName | ClassName | TeacherName | Grade |  
+-----+-----+-----+-----+  
| Arun        | X-A      | John        | A     |  
| Priya       | X-A      | John        | B     |  
| Rahul       | X-B      | Mary        | A     |  
| Sneha       | X-C      | David       | A+    |  
+-----+-----+-----+-----+  
4 rows in set (0.00 sec)  
  
mysql> SELECT D.DoctorName,  
-> COUNT(A.AppointmentID) AS TotalAppointments  
-> FROM Doctor D  
-> JOIN Appointment A  
-> ON D.DoctorID = A.DoctorID  
-> WHERE MONTH(A.AppointmentDate) = MONTH(CURDATE())  
-> AND YEAR(A.AppointmentDate) = YEAR(CURDATE())  
-> GROUP BY D.DoctorID, D.DoctorName  
-> HAVING COUNT(A.AppointmentID) > 10;  
Empty set (0.00 sec)  
  
mysql>  
mysql>
```

3. E-Commerce Sub-query

```
SELECT ProductName  
FROM Product  
WHERE ProductID NOT IN (  
    SELECT ProductID  
    FROM OrderDetails  
);
```

```

MySQL 8.0 Command Line CLI
-> FROM Student S
-> JOIN Class C
-> ON S.ClassID = C.ClassID
-> JOIN Teacher T
-> ON C.TeacherID = T.TeacherID;
+-----+
| StudentName | ClassName | TeacherName | Grade |
+-----+
| Arun        | X-A       | John        | A     |
| Priya       | X-A       | John        | B     |
| Rahul       | X-B       | Mary        | A     |
| Sneha       | X-C       | David       | A+    |
+-----+
4 rows in set (0.00 sec)

mysql> SELECT D.DoctorName,
-> COUNT(A.AppointmentID) AS TotalAppointments
-> FROM Doctor D
-> JOIN Appointment A
-> ON D.DoctorID = A.DoctorID
-> WHERE MONTH(A.AppointmentDate) = MONTH(CURDATE())
-> AND YEAR(A.AppointmentDate) = YEAR(CURDATE())
-> GROUP BY D.DoctorID, D.DoctorName
-> HAVING COUNT(A.AppointmentID) > 10;
Empty set (0.00 sec)

mysql>
mysql> SELECT ProductName
-> FROM Product
-> WHERE ProductID NOT IN (
-> SELECT ProductID
-> FROM OrderDetails
-> );
+-----+
| ProductName |
+-----+
| Monitor     |
+-----+
1 row in set (0.00 sec)

mysql>

```

4. Relational Algebra

$$\pi_{SID}(\sigma_{Course='DBMS'}(Enrollment)) \cap \pi_{SID}(\sigma_{Course='OS'}(Enrollment)) \setminus \pi_{\{SID\}}(\sigma_{Course='DBMS'}(Enrollment)) \cap \pi_{\{SID\}}(\sigma_{Course='OS'}(Enrollment)) \pi_{SID}(\sigma_{Course='DBMS'}(Enrollment)) \cap \pi_{SID}(\sigma_{Course='OS'}(Enrollment))$$

5. Payroll View

```

UPDATE Employee E
SET Salary = Salary * 1.10
WHERE Salary <
(
    SELECT AvgSalary
    FROM
    (
        SELECT DeptID, AVG(Salary) AS AvgSalary
        FROM Employee
        GROUP BY DeptID
    ) A
WHERE A.DeptID = E.DeptID
);

```

```
MySQL 8.0 Command Line Cli x + -
-> ) AS Temp
-> ) AS L
-> ON E.EmpID = L.EmpID
-> SET E.Salary = E.Salary * 1.10;
ERROR 1093 (HY000): You can't specify target table 'E' for update in FROM clause
mysql> CREATE TEMPORARY TABLE TempEmp AS
-> SELECT EmpID
-> FROM Employee E1
-> WHERE Salary < (
->     SELECT AVG(E2.Salary)
->     FROM Employee E2
->     WHERE E2.DeptID = E1.DeptID
-> );
Query OK, 2 rows affected (0.02 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> UPDATE Employee
-> SET Salary = Salary * 1.10
-> WHERE EmpID IN (
->     SELECT EmpID FROM TempEmp
-> );
Query OK, 2 rows affected (0.02 sec)
Rows matched: 2 Changed: 2 Warnings: 0

mysql> CREATE VIEW LowSalaryEmployees AS
-> SELECT E.EmpID,
->        E.EmpName,
->        E.Salary,
->        E.DeptID
-> FROM Employee E
-> WHERE Salary <
-> (
->     SELECT AVG(Salary)
->     FROM Employee E2
->     WHERE E2.DeptID = E.DeptID
-> );
ERROR 1050 (42S01): Table 'LowSalaryEmployees' already exists
mysql>
mysql> UPDATE Employee
-> SET Salary = Salary * 1.10
-> WHERE EmpID IN (
```

6. Banking JOIN Operations

```
SELECT C.CustomerName,
       A.AccountNumber,
       T.TransactionAmount,
       B.BranchName
FROM Customer C
JOIN Account A
ON C.CustomerID=A.CustomerID
JOIN Transactions T
ON A.AccountID=T.AccountID
JOIN Branch B
ON A.BranchID=B.BranchID;
```

```
MySQL 8.0 Command Line Cli
-> E.EmpName,
-> E.Salary,
-> E.DeptID
-> FROM Employee E
-> WHERE Salary <
-> (
-> SELECT AVG(Salary)
-> FROM Employee E2
-> WHERE E2.DeptID = E.DeptID
-> );
ERROR 1050 (42S01): Table 'LowSalaryEmployees' already exists
mysql>
mysql> UPDATE Employee
-> SET Salary = Salary * 1.10
-> WHERE EmpID IN (
-> SELECT EmpID
-> FROM LowSalaryEmployees
-> );
ERROR 1443 (HY000): The definition of table 'LowSalaryEmployees' prevents operation UPDATE on table 'Employee'.
mysql> SELECT C.CustomerName,
-> A.AccountNumber,
-> T.TransactionAmount,
-> T.TransactionDate,
-> B.BranchName
-> FROM Customer C
-> INNER JOIN Account A
-> ON C.CustomerID = A.CustomerID
-> INNER JOIN Transactions T
-> ON A.AccountID = T.AccountID
-> INNER JOIN Branch B
-> ON A.BranchID = B.BranchID;
+-----+-----+-----+-----+-----+
| CustomerName | AccountNumber | TransactionAmount | TransactionDate | BranchName |
+-----+-----+-----+-----+-----+
| Rahul        | SB1001        | 5000.00          | 2026-07-30      | Chennai    |
| Rahul        | SB1001        | 2000.00          | 2026-07-30      | Chennai    |
| Priya        | SB1002        | 8000.00          | 2026-07-30      | Coimbatore |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> |
```

7. Retail Store GROUP BY

SELECT Category,
SUM(SalesAmount) AS TotalSales
FROM Sales
GROUP BY Category
HAVING SUM(SalesAmount) > 10000;

```
MySQL 8.0 Command Line Cli
-> SET Salary = Salary * 1.10
-> WHERE EmpID IN (
-> SELECT EmpID
-> FROM LowSalaryEmployees
-> );
ERROR 1443 (HY000): The definition of table 'LowSalaryEmployees' prevents operation UPDATE on table 'Employee'.
mysql> SELECT C.CustomerName,
-> A.AccountNumber,
-> T.TransactionAmount,
-> T.TransactionDate,
-> B.BranchName
-> FROM Customer C
-> INNER JOIN Account A
-> ON C.CustomerID = A.CustomerID
-> INNER JOIN Transactions T
-> ON A.AccountID = T.AccountID
-> INNER JOIN Branch B
-> ON A.BranchID = B.BranchID;
+-----+-----+-----+-----+-----+
| CustomerName | AccountNumber | TransactionAmount | TransactionDate | BranchName |
+-----+-----+-----+-----+-----+
| Rahul        | SB1001        | 5000.00          | 2026-07-30      | Chennai    |
| Rahul        | SB1001        | 2000.00          | 2026-07-30      | Chennai    |
| Priya        | SB1002        | 8000.00          | 2026-07-30      | Coimbatore |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT Category,
-> SUM(SalesAmount) AS TotalSales
-> FROM Sales
-> GROUP BY Category
-> HAVING SUM(SalesAmount) > 10000;
+-----+-----+
| Category | TotalSales |
+-----+-----+
| Electronics | 13000.00 |
| Furniture | 12000.00 |
+-----+-----+
2 rows in set (0.00 sec)

mysql> |
```

8. Tuple Relational Calculus

$\{m \mid \text{Member}(m) \wedge (\forall g)(\text{Genre}(g) \rightarrow \exists b(\text{Borrow}(b) \wedge b.\text{MemberID} = m.\text{MemberID} \wedge b.\text{Genre} = g.\text{GenreName}))\}$
 $\{m \mid \text{Member}(m) \wedge (\forall g)(\text{Genre}(g) \rightarrow \exists b(\text{Borrow}(b) \wedge b.\text{MemberID} = m.\text{MemberID} \wedge b.\text{Genre} = g.\text{GenreName}))\}$
 $\{m \mid \text{Member}(m) \wedge (\forall g)(\text{Genre}(g) \rightarrow \exists b(\text{Borrow}(b) \wedge b.\text{MemberID} = m.\text{MemberID} \wedge b.\text{Genre} = g.\text{GenreName}))\}$

9. Inventory Constraints

```
CREATE TABLE Supplier(  
    SupplierID INT PRIMARY KEY,  
    SupplierName VARCHAR(50)  
);
```

```
CREATE TABLE Inventory(  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(50),  
    Quantity INT CHECK (Quantity >= 0),  
    SupplierID INT,  
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)  
);
```

```
MySQL 8.0 Command Line Cli  x  +  v  -  □  x  
-> INNER JOIN Transactions T  
-> ON A.AccountID = T.AccountID  
-> INNER JOIN Branch B  
-> ON A.BranchID = B.BranchID;  
+-----+-----+-----+-----+-----+  
| CustomerName | AccountNumber | TransactionAmount | TransactionDate | BranchName |  
+-----+-----+-----+-----+-----+  
| Rahul        | SB1001        | 5000.00           | 2026-07-30      | Chennai    |  
| Rahul        | SB1001        | 2000.00           | 2026-07-30      | Chennai    |  
| Priya        | SB1002        | 8000.00           | 2026-07-30      | Coimbatore |  
+-----+-----+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> SELECT Category,  
-> SUM(SalesAmount) AS TotalSales  
-> FROM Sales  
-> GROUP BY Category  
-> HAVING SUM(SalesAmount) > 10000;  
+-----+-----+  
| Category | TotalSales |  
+-----+-----+  
| Electronics | 13000.00 |  
| Furniture | 12000.00 |  
+-----+-----+  
2 rows in set (0.00 sec)  
  
mysql> CREATE TABLE Supplier (  
-> SupplierID INT PRIMARY KEY,  
-> SupplierName VARCHAR(50)  
-> );  
ERROR 1050 (42S01): Table 'supplier' already exists  
mysql>  
mysql> CREATE TABLE Inventory (  
-> ProductID INT PRIMARY KEY,  
-> ProductName VARCHAR(50),  
-> Quantity INT CHECK (Quantity >= 0),  
-> SupplierID INT,  
-> FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)  
-> );  
ERROR 1050 (42S01): Table 'inventory' already exists  
mysql> |
```

10. HR System

Create View for Monthly Attendance

```
CREATE VIEW MonthlyAttendance AS
SELECT EmpID,
       COUNT(*) AS TotalPresent
FROM Attendance
WHERE Status = 'Present'
GROUP BY EmpID;
```

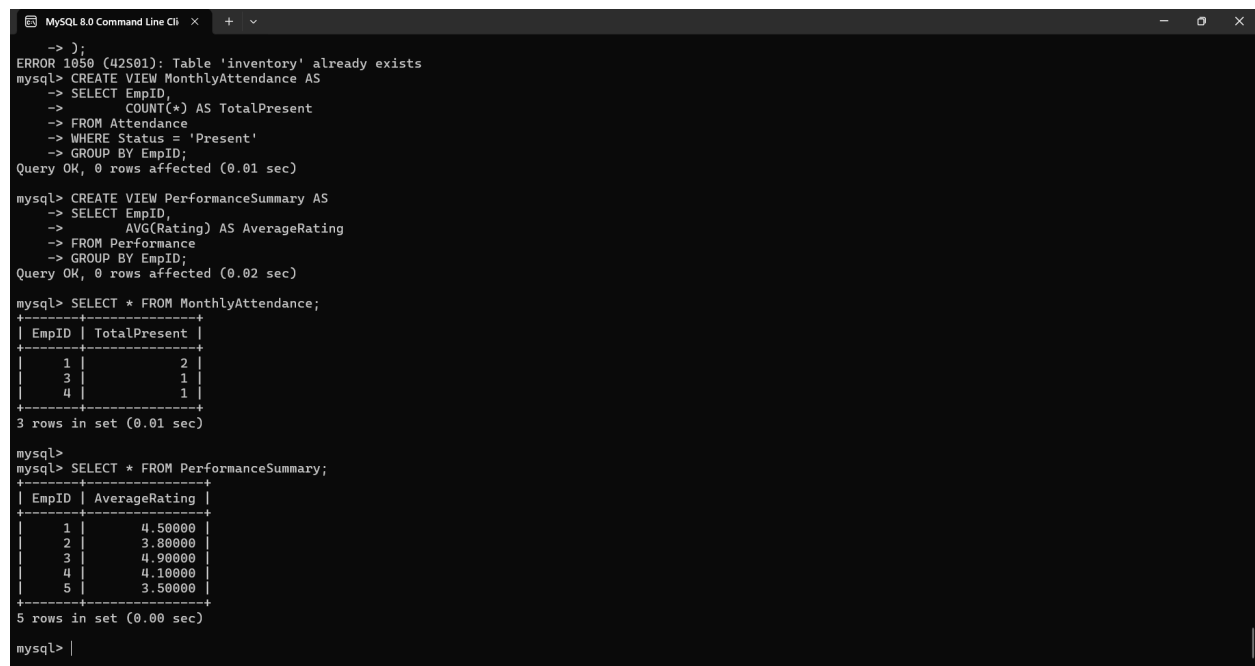
Create View for Performance Summary

```
CREATE VIEW PerformanceSummary AS
SELECT EmpID,
       AVG(Rating) AS AverageRating
FROM Performance
GROUP BY EmpID;
```

Query the Views

```
SELECT * FROM MonthlyAttendance;
```

```
SELECT * FROM PerformanceSummary;
```



```
MySQL 8.0 Command Line Cli x + v
-> );
ERROR 1050 (42S01): Table 'inventory' already exists
mysql> CREATE VIEW MonthlyAttendance AS
-> SELECT EmpID,
->        COUNT(*) AS TotalPresent
-> FROM Attendance
-> WHERE Status = 'Present'
-> GROUP BY EmpID;
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE VIEW PerformanceSummary AS
-> SELECT EmpID,
->        AVG(Rating) AS AverageRating
-> FROM Performance
-> GROUP BY EmpID;
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT * FROM MonthlyAttendance;
+-----+-----+
| EmpID | TotalPresent |
+-----+-----+
| 1     | 2           |
| 3     | 1           |
| 4     | 1           |
+-----+-----+
3 rows in set (0.01 sec)

mysql>
mysql> SELECT * FROM PerformanceSummary;
+-----+-----+
| EmpID | AverageRating |
+-----+-----+
| 1     | 4.50000      |
| 2     | 3.80000      |
| 3     | 4.90000      |
| 4     | 4.10000      |
| 5     | 3.50000      |
+-----+-----+
5 rows in set (0.00 sec)

mysql> |
```